

Some History

- For a very long time, Computer Vision was gaining more attention when Deep Learning was starting to get popular. You probably heard of AlexNet, ResNet, RCNN or YOLO (I am surprised it is already at YOLO v11).
- A famous paper changed the landscape at 2017, **"Attention Is All You Need"**. Since then, Transformer has become the most popular architecture for many tasks.
- Most people would probably know Large Language Models (LLMs) are mostly based on Transformer. However, it is also used in Computer Vision, and is becoming more and more important as Multimodal LLMs becoming what people want naturally.
 - Similarly, Diffusion model is not only used in image generation, but also in text. Will save it for some other time :)
- Let's talk about **Vision Transformer (ViT)** today!

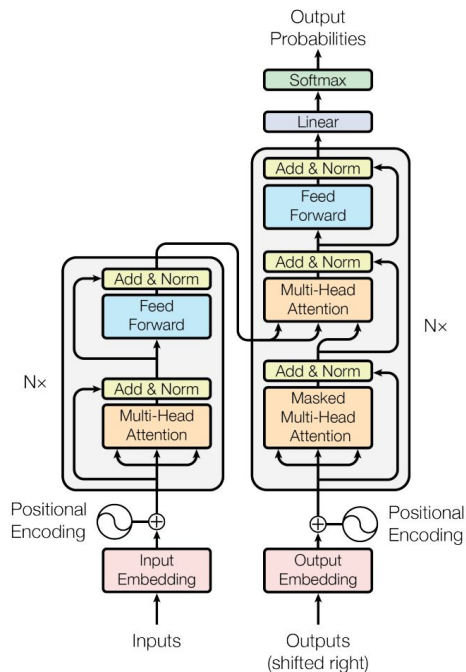


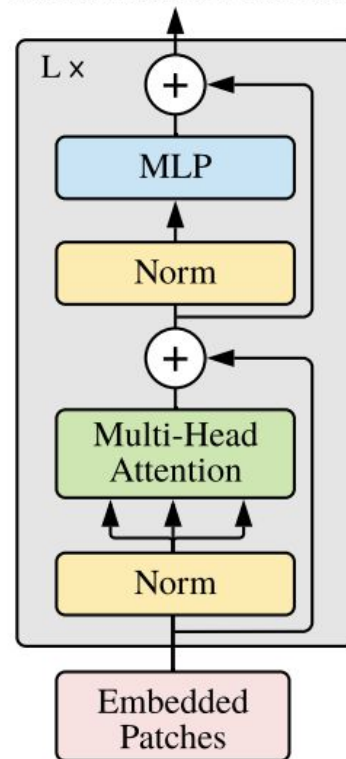
Figure 1: The Transformer - model architecture.

What Is Vision Transformer?

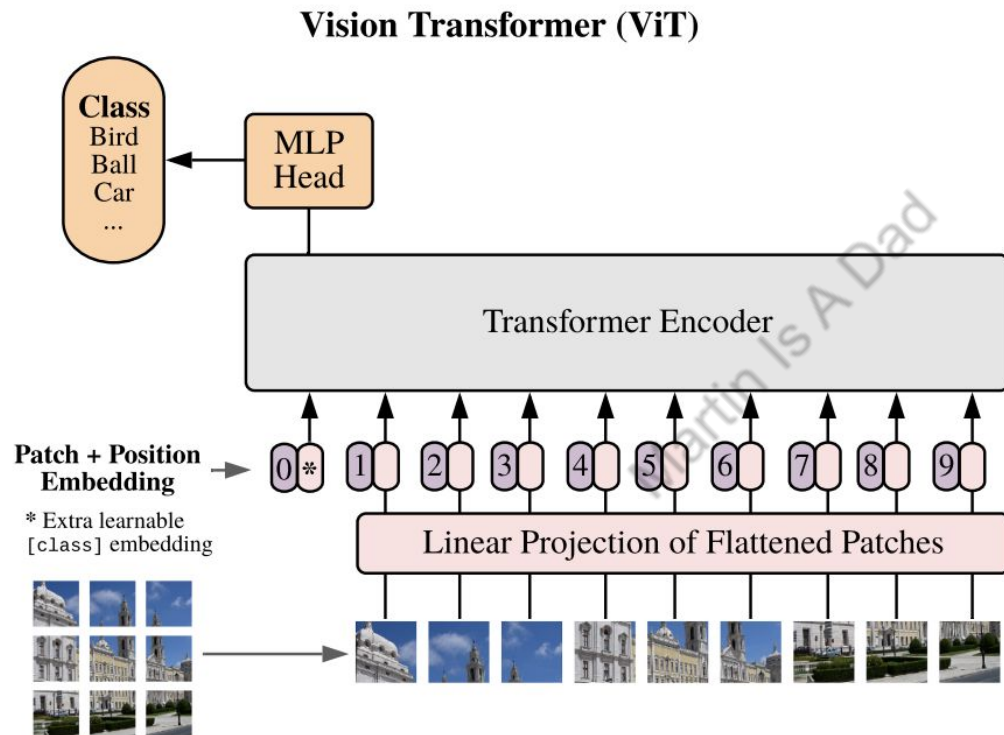


Martin Is A Dad

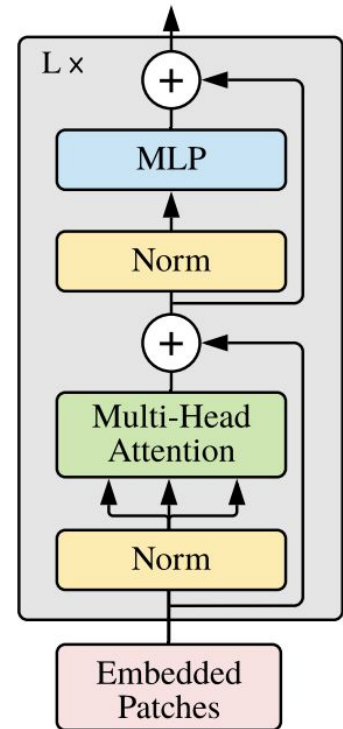
Transformer Encoder



What Is Vision Transformer?



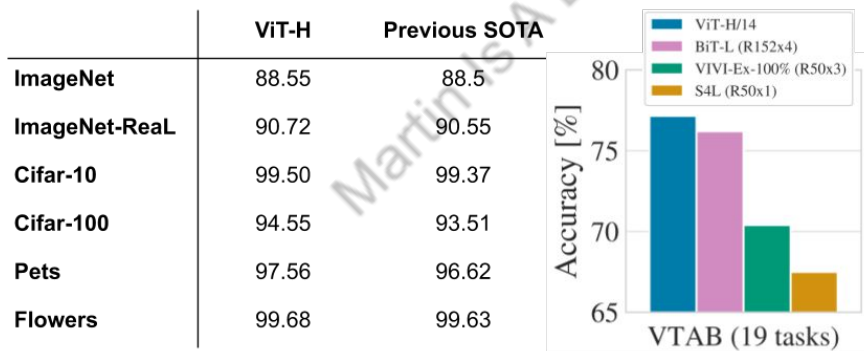
Transformer Encoder



What Is Vision Transformer?

Vision Transformer (ViT) is a vision model similar to the Transformer architecture originally designed for text-based tasks.

- ViT represents an input image as a sequence of image patches, similar to the sequence of word embeddings used when applying Transformers to text, and directly predicts class labels for the image.
- ViT demonstrates excellent performance when trained on sufficient data, outperforming a comparable state-of-the-art CNN with four times fewer computational resources, on popular classification tasks when it first came out at 2020.



- In the more recent Multimodal LLM world, ViTs play a pivotal role by enabling these models to seamlessly integrate and understand visual information alongside textual data. This integration is crucial for tasks that require a holistic understanding of both modalities.

ViT Walkthrough: Image Tokenization

Treating Images as Sequences of Patches:

- Transformer models process inputs as tokens. Can we afford treating each pixel as a token? Recall that the computational complexity of calculating the attention matrix is N^2 where N is the sequence length. If we treat each pixel as a separate token, then assuming a relatively small image size of 100×100 , the attention matrix will be of size 10000×10000 . Too expensive even with GPUs.
- Alternative is a patch of some size, say 16×16 , as one token. A RGB image of size $W \times H \times 3$ is divided up into patches, each of size $w \times h \times 3$. Each patch is flattened and passed through a dense/fully connected/ffn layer without activation for flattening. This dense embedding layer transforms the patch into a hidden, learned representation of dimension d_{in} .

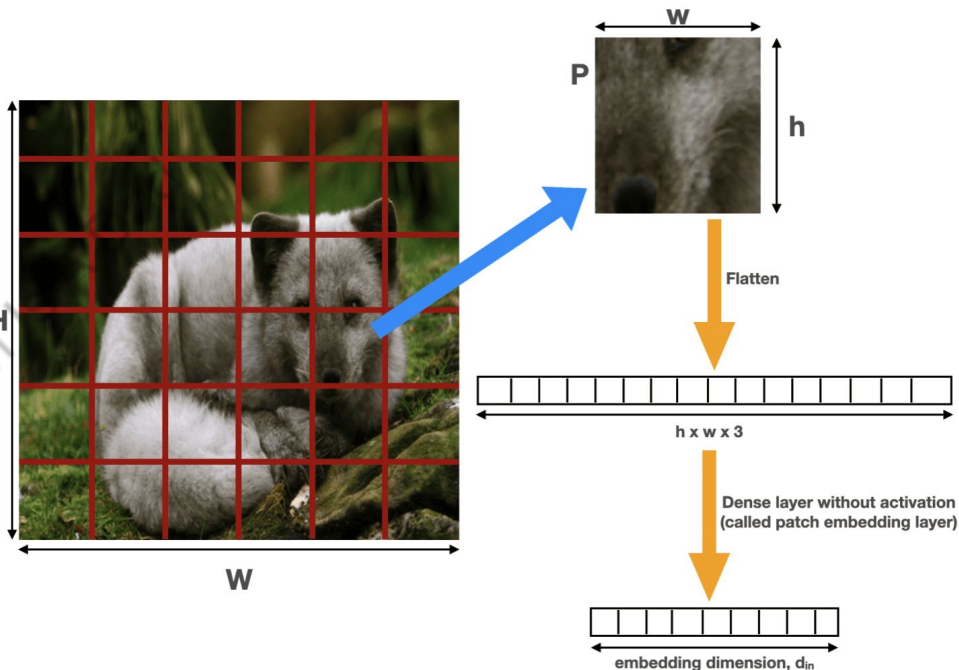


Image from <https://learnopencv.com/the-future-of-image-recognition-is-here-pytorch-vision-transformer/>

ViT Walkthrough: Image Tokenization

Treating Images as Sequences of Patches:

- As described, the initial and crucial step in a ViT is to transform a 2D image into a sequence of 1D tokens, conceptually similar to words in a sentence.
- This is achieved by splitting the input image into a grid of fixed-size, typically non-overlapping, square patches. For instance, a 224x224 pixel image might be divided into 16x16 pixel patches. Each of these 2D image patches is then flattened into a 1D vector of pixel values, effectively becoming a "visual word" or "token" in the sequence that the Transformer can process.

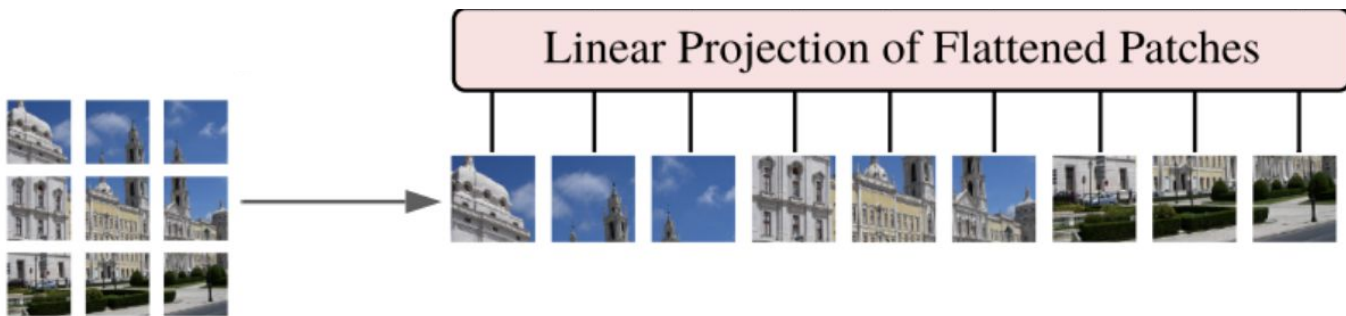


ViT Walkthrough: Patch Embedding

Linear Projection of Patches:

- Once flattened, each patch vector undergoes a linear projection into a higher-dimensional embedding space. This linear transformation converts the raw pixel data of each patch into a dense vector representation, making it suitable for the Transformer encoder's operations.
- To facilitate image classification, a special, learnable "classification token" (CLS token) is typically prepended to this sequence of patch embeddings. This token is designed to aggregate global information from all image patches as it passes through the Transformer layers, and its final output state is then used for the image classification decision.

$$\mathbf{z}_0 = [\underbrace{\mathbf{x}_{\text{class}}}_{\text{CLS Token}}; \underbrace{\mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}}_{\text{Linear Transformation on flatten image patches}}] + \underbrace{\mathbf{E}_{\text{pos}}}_{\text{Position Encoding}}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times D}$$



ViT Walkthrough: CLS Token

- Don't get misguided by the name "Classification Token", this token is **not** only useful in classification tasks
- The CLS token is a **special, learnable embedding** that is prepended to the sequence of image patch embeddings before they are fed into the Transformer encoder. Conceptually it serves as a representation of the entire input sequence or sentence.
- In a standard ViT for image classification, CLS token's primary function is to **aggregate global information** from all the individual image patches.
 - CLS Token learns to weigh the importance of different visual regions across the entire image. The final output state of this CLS token, after it has interacted with all the patch embeddings through the self-attention layers, is then typically passed to a MLP. Essentially, the CLS token becomes a condensed, holistic representation of the entire image, suitable for a single classification decision.

$$\mathbf{z}_0 = [\underbrace{\mathbf{x}_{\text{class}}}_{\text{CLS Token}}; \underbrace{\mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}}_{\text{Linear Transformation on flatten image patches}}] + \underbrace{\mathbf{E}_{pos}}_{\text{Position Encoding}}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$

ViT Walkthrough: CLS Token

- In **Multimodal LLMs**, CLS token acts as a powerful aggregator and importance indicator for visual information within ViT, enabling efficient visual understanding in complex multimodal systems.
 - **Visual Importance Indicator**: CLS token can learn which visual tokens (image patches) are most important for the model's overall understanding and for generating relevant textual responses. Its attention scores on the visual tokens can serve as a direct indicator of their importance.
 - **Visual Token Compression**: Methods like Visual Token Compression utilize the attention scores of the CLS token on visual tokens to prune redundant visual information.
 - **High-Resolution Image Processing**: Attention patterns of the CLS token are used to assess the visual content of different image partitions. By aggregating CLS-attention (across multiple heads) from the final layer as a feature importance score, the model can intelligently select the most informative visual tokens within an allocated budget.
 - **Semantic Awareness**: Attention of CLS token has been shown to correlate with visual content, helping to identify main objects and distinguish them from irrelevant backgrounds within an image.

$$\mathbf{z}_0 = [\mathbf{x}_{\text{class}}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \quad \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$

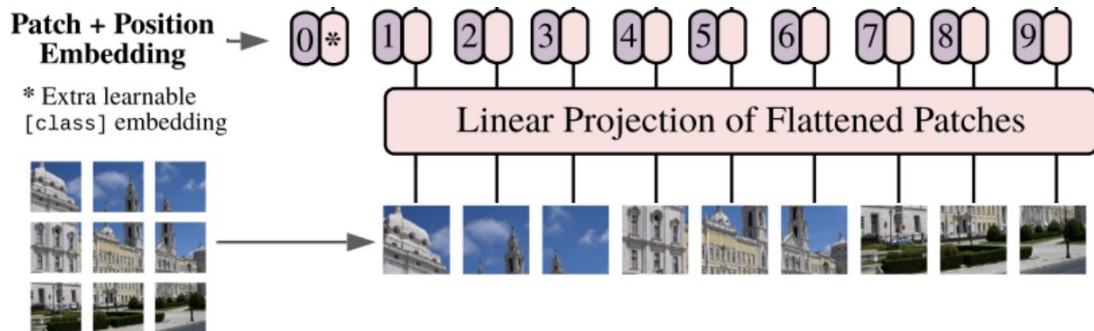
CLS Token Linear Transformation on
flatten image patches Position
Encoding

ViT Walkthrough: Positional Encoding

Retaining Spatial Information:

- A critical aspect of the Transformer architecture is its inherent permutation-invariance; it processes sequences without an intrinsic understanding of the order or spatial arrangement of its input tokens. In text based Transformer based LLMs, we use sinusoidal positional encoding, or RoPE to encode positional information.
- Similarly in ViT, to reintroduce this vital spatial context for images, positional encodings are added to the patch embeddings. These encodings provide the model with information about the original position of each patch within the image.

$$\mathbf{z}_0 = [\underbrace{\mathbf{x}_{\text{class}}}_{\text{CLS Token}}; \underbrace{\mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}}_{\text{Linear Transformation on flatten image patches}}] + \underbrace{\mathbf{E}_{pos}}_{\text{Position Encoding}}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \quad \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$

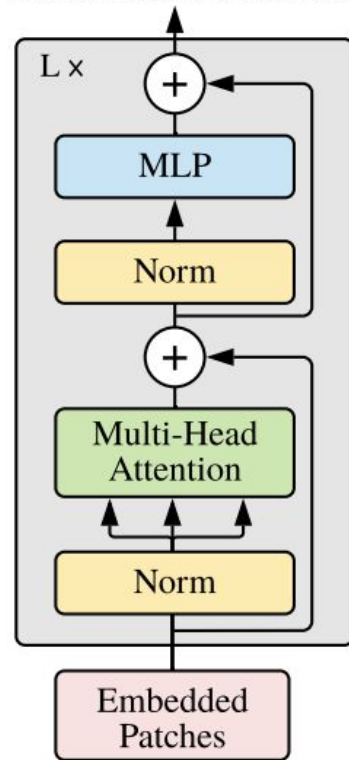


ViT Walkthrough: Encoder Block

The Core Mechanism for Global Context - Multi-Head Self-Attention:

- The combined sequence of patch embeddings, positional encodings, and the CLS token is fed into a standard Transformer encoder, which is composed of multiple identical layers.
- The heart of each Transformer layer is the Multi-Head Self-Attention mechanism. It enables each patch (token) to dynamically weigh the importance of, all other patches in the image. This global connectivity allows the model to capture complex relationships and long-range dependencies across the entire visual input, a key differentiating factor from CNNs.
- The "multi-head" aspect means that the attention mechanism is performed multiple times in parallel, each with distinct learning target. This parallel processing allows the model to learn different types of relationships and enrich the overall representation.

Transformer Encoder

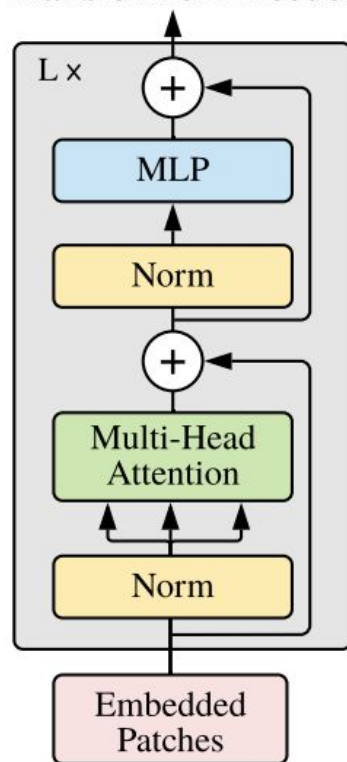


ViT Walkthrough: Encoder Block

Beyond Multi-Head Self-Attention:

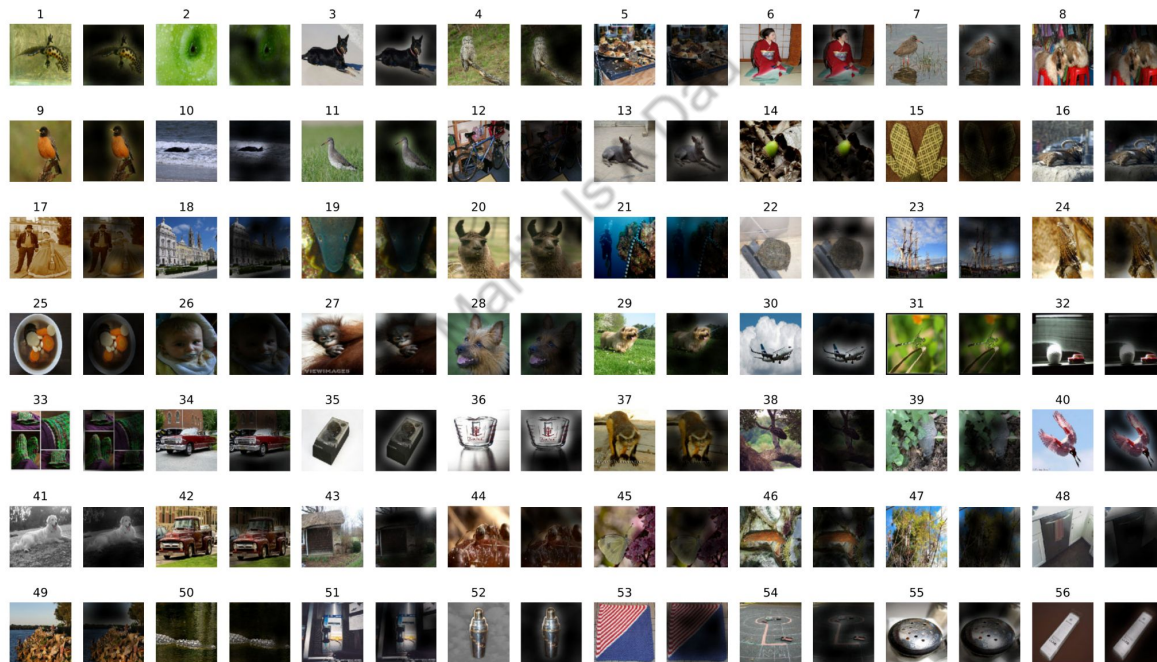
- Beyond the MHSA module, each Transformer encoder block typically includes a Layer Normalization layer and a Multi-Layer Perceptron (MLP) feed-forward layer. Residual connections (also known as skip connections) are employed around each sub-layer to facilitate gradient flow and stabilize training in deep networks.
- For image classification, the final output of the Transformer encoder, specifically the transformed CLS token, is then passed to a simple classification head (e.g., another MLP) to predict the image's category.
- Architectural innovations within these blocks are ongoing, such as replacing standard MLP sub-layers with Kolmogorov–Arnold Network (KAN) expansions to potentially capture more complex nonlinear dependencies. Furthermore, efficient attention mechanisms like Flash Attention are being integrated to optimize the computational efficiency of the attention process. Let's save KAN for another episode :)

Transformer Encoder



ViT Walkthrough: Attention Map

An attention map is a visualization that reveals which parts of an image the model focuses on when making a classification decision. It essentially highlights the importance of different image patches, or "tokens," in the model's prediction.



ViT vs. CNN

Advantages of ViTs

- **Less Inductive Biases:** Inductive bias in machine learning refers to the assumptions a learning algorithm makes to generalize from observed training data to unseen data. This can be bad when the assumption is wrong.
 - CNNs are inherently designed with strong inductive biases that reflect assumptions about the nature of image data. These include locality, where pixels are strongly correlated with their immediate neighbors; translational equivariance, where a pattern recognized in one part of an image will be recognized if it shifts to another. These biases are hardwired into their convolutional kernels and pooling layers, making them highly efficient at extracting local features and patterns.
 - In contrast, ViTs assume minimal prior knowledge or inductive bias about the spatial structure of images. They treat images as flat sequences of patches, relying solely on the self-attention mechanism to learn all relationships from scratch.
- **Dynamic Feature Computation:** A significant advantage of self-attention over convolutional processes is its capability to dynamically compute filters for every input sequence. This allows the model to adapt its feature extraction to the particular context of the input data, unlike CNNs' static, pre-learned weights.

ViT vs. CNN

Advantages of ViTs

- **Superior Global Context Modeling:** ViTs excel at capturing long-range dependencies and global relationships across an entire image. Their self-attention mechanism allows every patch to interact with and weigh the importance of every other patch, providing a holistic view of the image that CNNs struggle to achieve.
- **Enhanced Scalability and Generalization:** When pre-trained on sufficiently large datasets, ViTs demonstrate remarkable scalability, flexibility, and superior generalization capabilities. They perform competitively or even outperform state-of-the-art CNNs on challenging benchmarks like ImageNet. Their ability to handle distribution shifts, occlusions, and perturbations makes them highly robust for real-world applications.

ViT vs. CNN

Limits of ViTs

- **Significant Data Hunger:** Like other Transformer based models, ViT depend on very large datasets for pre-training to achieve competitive performance. Without such extensive data, ViTs may underperform compared to CNNs, which can learn effectively from smaller datasets due to their stronger inductive biases.
- **High Computational and Memory Demands:** The quadratic complexity of the self-attention mechanism with respect to the number of tokens (which directly correlates with image resolution) leads to substantial computational overhead (FLOPs) and high memory consumption.
- **Increased Training Difficulty:** Training ViTs can be more challenging and time-consuming than training CNNs. They often require more epochs to converge and are sensitive to optimization strategies.
- **Interpretability Challenges:** The intricate multi-head attention mechanism, particularly the complex blending of attention weights from each layer, makes ViTs less transparent than CNNs.
- **Fixed-Sized Input Tokens/Embedding:** While ViTs process images as patches, the initial tokenization often assumes fixed-sized patches, which can sometimes limit their flexibility compared to continuous input processing.

MultiModal LLM Key Components

Component	Modality Encoders	Input Projector	LLM Backbone	Output Projector	Modality Generator
Primary Function	Transform raw data from various modalities (image, audio, 3D, etc.) into numerical feature representations (embeddings).	Align encoded features from different modalities into a common space, typically compatible with the LLM backbone's input. Typically using MLPs, cross attentions, Q-Formers etc.	Serves as the central reasoning and language processing engine, integrating aligned multimodal information. Usually pretrained LLMs, say GPT, Gemini, LLaMA etc.	Maps LLM outputs (e.g., signal tokens for generation) into features suitable for modality-specific generators. Typically MLPs or Transformers.	Produces outputs in non-textual modalities (e.g., images, audio) based on input from the Output Projector. This can be StableDiffusion, AudioLDM etc

ViT in MultiModal LLM - Image Generator

- Recall that Diffusion (DDPM, DDIM, LDM etc) uses UNet as it's architecture, which contains convolution layers to extract image features. More details take a look at my Diffusion Deep Dive.
- We already know ViT surpass CNN in many aspects. Replacing UNet's convolution layers with MLP + Attention will get us U-ViT.
- This improvement is used in recent generative diffusion models (Imagen etc)

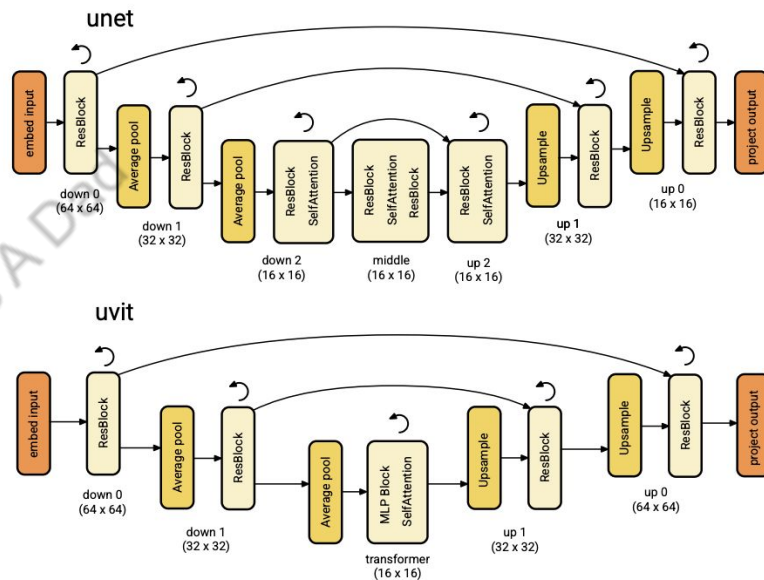
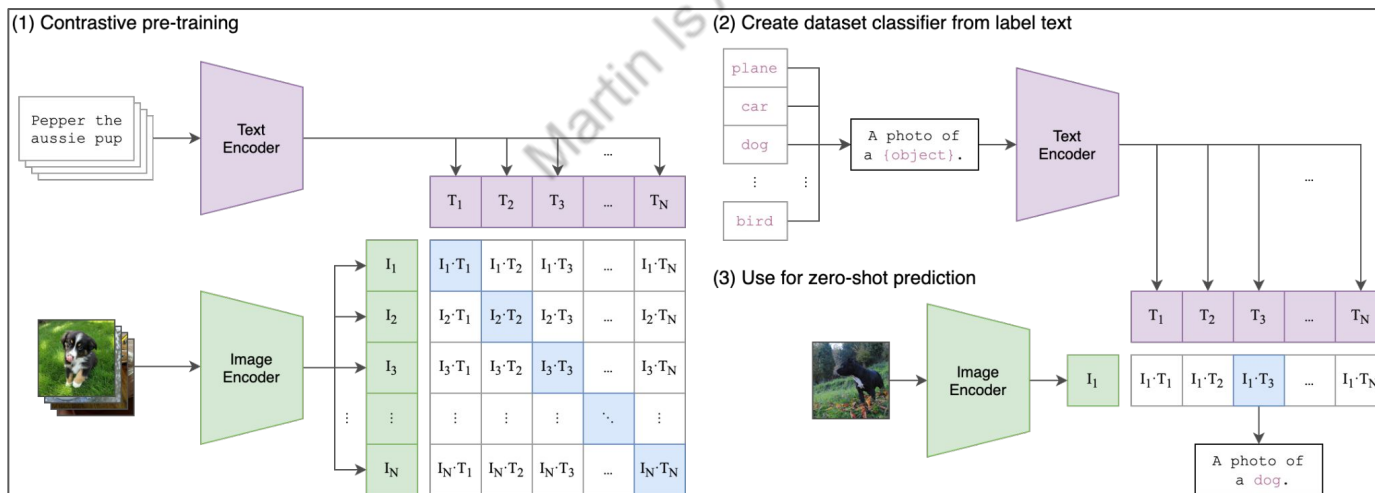


Figure 7: The difference between the U-Net and U-ViT architecture. In essence, the convolutional layers are replaced by MLP blocks on levels with self-attention. These now form transformer blocks which are connected via residual connections, only the ResBlocks on higher levels use skip connections. Circular arrows denote that such a block can be repeated multiple times.

ViT in MultiModal LLM - Image Encoder

CLIP (Contrastive Language-Image Pre-training) is designed to learn visual concepts from natural language supervision, and is an important framework for Multimodal LLM.

- Unlike traditional image classification models that are trained on fixed categories (e.g., ImageNet's 1000 classes), CLIP learns an "open set" of visual concepts by associating images with their natural language descriptions.
- This allows for remarkable zero-shot capabilities, meaning it can classify images or understand visual concepts it has never explicitly seen during training, simply by being given a textual description.



CLIP TL;DR

Key Components

- **Image Encoder:** This neural network takes an image as input and transforms it into a numerical representation called an "embedding" or "feature vector." This embedding captures the salient visual features of the image. **This is where ViT shines.**
- **Text Encoder:** This neural network takes a piece of text (e.g., a caption, a phrase) as input and transforms it into a numerical representation (an embedding) that captures its semantic meaning.
- **Shared Embedding Space:** The crucial innovation of CLIP is that both the image encoder and the text encoder are trained to map their respective inputs into a shared, high-dimensional embedding space. In this space, embeddings of semantically similar images and text are close together, while those of dissimilar pairs are far apart.

Training Essence: CLIP is trained on a massive dataset of image-text pairs. The training process involves a contrastive learning objective. It learns robust and generalizable representations for both modalities. It effectively learns to tell "what goes with what" in the visual and linguistic world.

- **Positive Pairs:** For a given batch of image-text pairs, the model considers the actual matching image and text as a "positive pair." It aims to maximize the cosine similarity (a measure of how similar two vectors are) between their embeddings in the shared space.
- **Negative Pairs:** All other image-text combinations within the batch (i.e., a given image with a non-matching caption from the batch, or a given caption with a non-matching image) are treated as "negative pairs." The model aims to minimize the cosine similarity between their embeddings.

Why Is ViT Great For CLIP

- **Global Context:** The self-attention mechanism in ViTs allows CLIP to learn global relationships and dependencies within an image, which is essential for understanding complex scenes and associating them with rich, descriptive text.
- **Scalability:** Like LLMs, ViTs benefit from scaling up model size and training data. Larger ViT models trained on more data tend to yield better performance.
- **Unified Architecture:** Using a Transformer-based architecture for both vision and language (even if they are separate encoders in CLIP) facilitates the alignment of their embeddings in the shared space, as they share similar underlying computational principles.

ViT as Encoder in MultiModal LLM

